

《面向对象程序设计》之——

继 承

连 盛

shenglian@fzu.edu.cn

计算机与大数据学院

福州大学

讲授内容

- 继承和派生的基本概念
- 继承的定义
- 在派生类中重定义基类的函数
- 基类和派生类对象的转换
- 类指针的使用
- 继承关系中构造函数、析构函数之间的关系
- 多重继承
- 复合与继承的关系

继承的概念

■ 实体之间的关系

■ 有 (has-a) ——类的复合

- 班级和学校，班级不是学校，学校拥有班级，班级是学校的一部分

■ 是 (is-a) ——类的继承

- 中学和学校，中学是学校，学校不能拥有中学，中学是学校的一个特例，是某一种学校

继承性是面向对象程序设计中最重要机制之一。这种机制提供了无限重复利用程序资源的一种途径。通过C++语言中的继承机制，可以扩充和完善旧的程序设计以适应新的需求。这样不仅可以节省程序开发的时间和资源，并且为未来程序增添了新的资源。

```
class Student
```

```
{ int num;
```

```
char name[30];
```

```
char sex;
```

```
public:
```

```
void display( ) //对成员函数display的定义
```

```
{cout<<"num: "<<num<<endl;
```

```
cout<<"name: "<<name<<endl;
```

```
cout<<"sex: "<<sex<<endl; }
```

```
};
```

class Student1

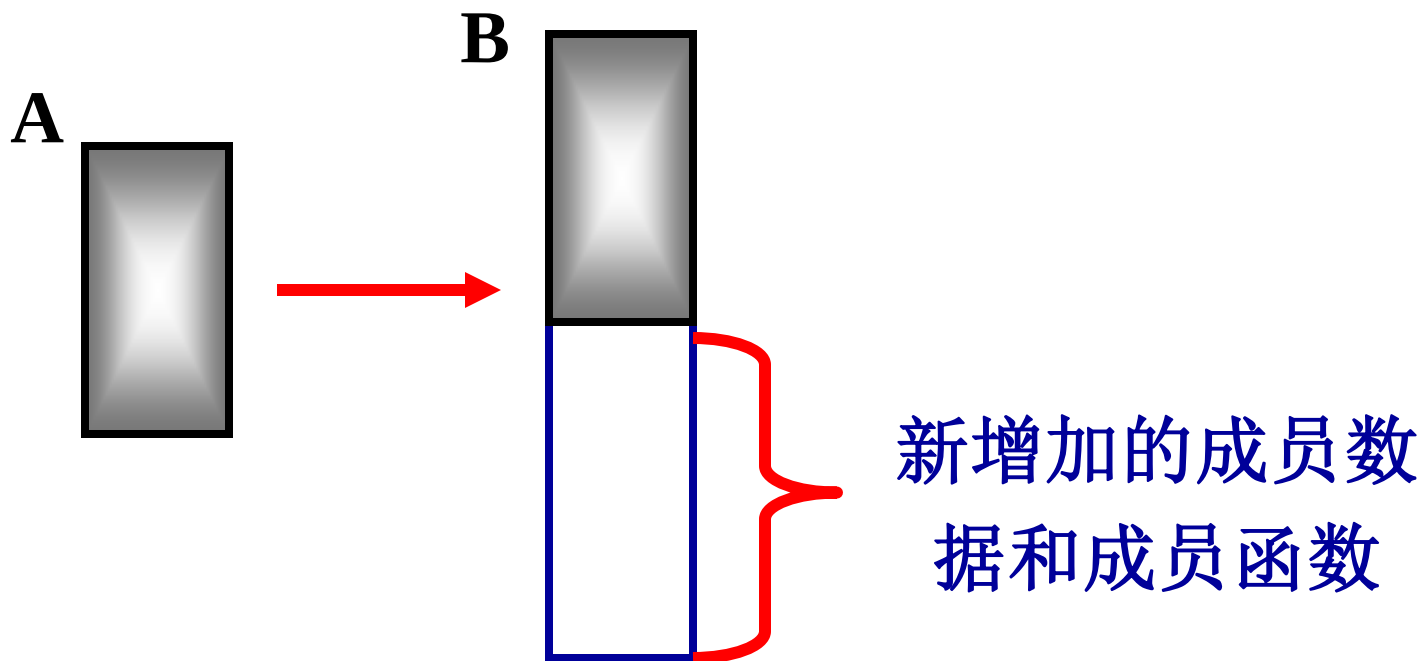
```
{  int num;          //此行原来已有  
    char name[20];    //此行原来已有  
    char sex;         //此行原来已有  
  
    int age;  
    char addr [20] ;  
  
public:  
  
    void display( ) ;    //此行原来已有  
    {cout<<"num: "<<num<<endl;  //此行原来已有  
      cout<<"name: "<<name<<endl; //此行原来已有  
      cout<<"sex: "<<sex<<endl;   //此行原来已有  
      cout<<"age: "<<age<<endl;  
      cout<<"address: "<<addr<<endl;}  
};
```

利用原来定义的类Student作为基础，再加上新的内容即可，以减少重复的工作量。 C++提供的继承机制就是为了解决这个问题。

在C++中所谓“继承”就是在一个已存在的类的基础上建立一个新的类。已存在的类称为“基类(base class)”或“父类(father class)”。新建立的类称为“派生类(derived class)”或“子类(son class)”。

```
class Student1: public Student//声明基类是Student
{private:
    int age;    //新增加的数据成员
    string addr; //新增加的数据成员
public:
    void display_1( ) //新增加的成员函数
    { cout<<"age: "<<age<<endl;
      cout<<"address: "<<addr<<endl;
    }
};
```


类A派生类B：类A为基类，类B为派生类。



在C++语言中，一个派生类可以从一个基类派生，也可以从多个基类派生。从一个基类派生的继承称为单继承；从多个基类派生的继承称为多继承。

通过继承机制，可以利用已有的数据类型来定义新的数据类型。所定义的新的数据类型不仅拥有新定义的成员，而且还同时拥有旧的成员。我们称已存在的用来派生新类的类为基类，又称为父类。由已存在的类派生出的新类称为派生类，又称为子类。

- 在建立派生类的过程中，基类本身不会有任何改变
- 派生类则除了继承基类的所有可引用的成员变量和成员函数外，还可另外定义本身的成员变量和处理这些变量的函数
- 由于派生类可继承基类的成员变量和成员函数，因此在基类中定义好的数据和函数等的程序代码可重复使用，这样可以提高程序的可靠性。

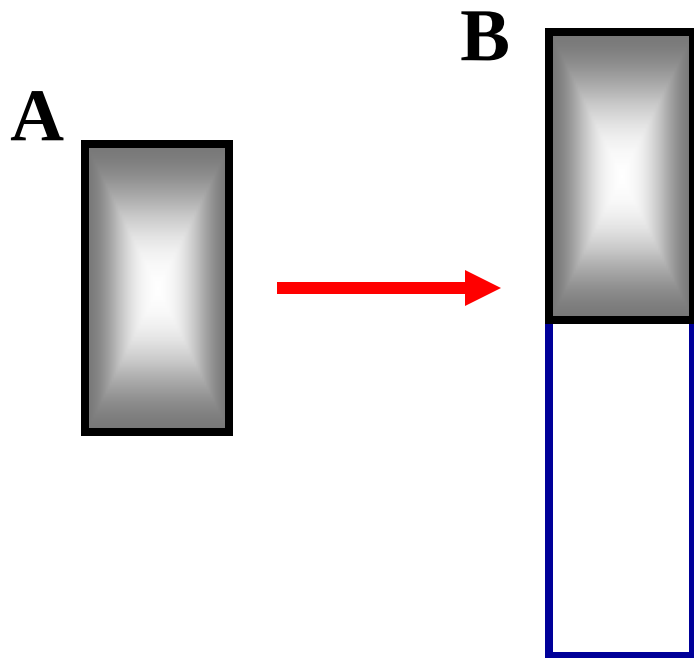
当从已有的类中派生出新的类时，可以对派生类做以下几种变化：

- 1、可以继承基类的成员数据或成员函数。
- 2、可以增加新的成员变量。
- 3、可以增加新的成员函数。
- 4、可以重新定义已有的成员函数。
- 5、可以改变现有的成员属性。

在C++中有二种继承：单一继承和多重继承。当一个派生类仅由一个基类派生时，称为单一继承；而当一个派生类由二个或更多个基类所派生时，称为多重继承。

- 继承的例子无处不在
 - 家具和柜子
 - 建筑物和房子
 - 道路和高速公路
 - 动物和猫
 - 电视和彩色电视
 - 电脑和笔记本电脑

类A派生类B：类A为基类，类B为派生类。



class B: **public** A{...};

class B: **protected** A{...};

class B: **private** A{...};

class B: A {...};

A为私有派生

但派生并不是简单的扩充，有可能改变基类的性质。

继承的定义

- 继承的三种方式
 - 公有继承 (public)
 - 受保护继承 (protected)
 - 私有继承 (private)
 - 在不同方式下, 基类中的成员被派生类继承后对外的可见性会有所不同

继承的定义

- 公有继承 (public)
 - 基类中公有成员和受保护成员被继承后可见性不变
- 受保护继承 (protected)
 - 基类中公有成员和受保护成员被继承后都是受保护的
- 私有继承 (private)
 - 基类中公有成员和受保护成员被继承后都是私有的

派生类成员的访问属性

✦ 分情况处理

- 基类的成员函数访问基类成员 ✓
- 派生类成员函数访问派生类自己增加的成员 ✓
- 基类的成员函数访问派生类的成员 ✗
- 派生类外访问派生类的成员
 - 根据成员的访问属性判定能否由类外访问该成员
- 派生类的成员函数访问基类的成员
- 派生类外访问基类的成员
- 核心问题：如何确定基类成员在派生类中的访问属性
 - 基类成员声明的访问属性
 - 派生类对基类的继承方式

三种继承方式下派生类中基类成员的访问控制权限

继承方式 基类成员	<u>公有继承</u>	<u>私有继承</u>	<u>保护继承</u>
公有成员	公有	私有	保护
私有成员	派生类成员不可访问	派生类成员不可访问	派生类成员不可访问
保护成员	保护	私有	保护

从一个基类派生一个类的一般格式为：

```
class ClassName:<Access>BaseClassName
```

```
{
```

派生类名

继承方式

基类名

```
private:
```

```
.....;    //私有成员说明
```

```
public:
```

```
.....;    //公有成员说明
```

```
protected:
```

```
.....;    //保护成员说明
```

```
}
```

派生类中新增加的成员

公有派生

公有派生，派生类中保持基类的成员特性

```
class ClassName: public BaseClassName
```

公有派生时，基类中所有成员在派生类中保持各个成员的访问权限。

基类成员属性	派生类中	派生类外
公有	可以引用	可以引用
保护	可以引用	不可引用
私有	不可引用	不可引用

基类: public: 在派生类和类外可以使用

protected: 在派生类中使用

private: 不能在派生类中使用

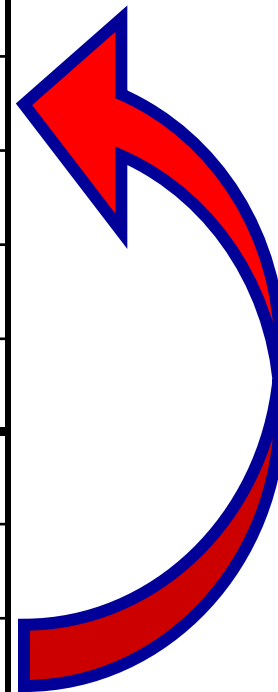
A

x(私有)
y(保护)
z(公有)
A()
Getx()
Gety()
ShowA()

B

x(私有)
y(保护)
z(公有)
A()
Getx()
Gety()
ShowA()
m(私有)
n(私有)
B()
Show()
Sum()

public



x在类B新增加的成员中不能直接调用

y在类B中可以调用

z在整个文件中可以调用

对类B的对象初始化即是对x,y,z,m,n等全部成员的初始化

```
class A {  int x;
protected:  int y;
public:    int z;

  A(int a,int b,int c){x=a;y=b;z=c;}//基类初始化
  int Getx(){return x;}    //返回x
  int Gety(){return y;}    //返回y
  void ShowA(){cout<< "x="<<x<<"\t"<<"y="<<y<<"\t"<<"z="<<z<<"\n";}
};
```

因为y是基类保护，所以在派生类中可以直接引用。而在类外不可直接引用。

```
class B:public A{
    int m,n;
public:  B(int a,int b,int c,int d,int e):A(a,b,c){m=d;n=e;}
    void Show(){cout<<"m="<<m<<"\t"<<"n="<<n<<"\n";
    cout<<"x="<<Getx()<<"\t"<<"y="<<y<<"\t"<<"z="<<z<<"\n"; }
    int Sum(){return ( Getx()+y+z+m+n);}
};
```

公有派生

对基类初始化

因为z是基类公有，所以在派生类中和类外均可直接引用。

因为x是基类私有，所以在派生类和类外中不能直接引用

```
void main(void)
{  B b1(1,2,3,4,5);
  b1.ShowA();    b1.Show();
  cout<< "Sum="<<b1.Sum()<<"\n";cout<<"x="<<b1.Getx()<<"\t";
  cout << "y=" <<b1.Gety()<<"\t";    cout << "z="<<b1.z<<"\n";}
```




私有派生

私有派生，派生类中基类公有和保护成员成为私有

```
class ClassName: private BaseClassName
```

私有派生时，基类中公有成员和保护成员在派生类中均变为私有的，在派生类中仍可直接使用这些成员，基类中的私有成员，在派生类中不可直接使用。

基类成员属性	派生类	派生类外
公有	可以引用	不可引用
保护	可以引用	不可引用
私有	不可引用	不可引用

- 基类: public: (变为私有)在派生类中使用，类外不可使用
- protected: (变为私有) 在派生类中使用，类外不可使用
- private: 不能在派生类中和类外使用

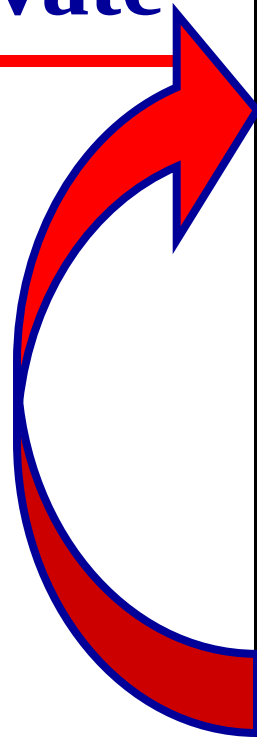
A

x(私有)
y(保护)
z(公有)
A()
Getx()
Gety()
ShowA()

B

x(私私有)
y(私有)
z(私有)
A()
Getx()
Gety()
ShowA()
m(私有)
n(私有)
B()
Show()
Sum()

private



均为私有
类B 外不
能引用

x在类B新增加的成员中
不能直接调用

y在类B中可以调用

z在类B中可以调用

对类B的对象初始化即是对x,y,z,m,n等全部成员的初始化

```
class A {  int x;
protected:  int y;
public:    int z;

  A(int a,int b,int c){x=a;y=b;z=c;}//基类初始化
  int Getx(){return x;}    //返回x
  int Gety(){return y;}    //返回y
  void ShowA(){cout<< "x="<<x<<"\t"<<"y="<<y<<"\t"<<"z="<<z<<"\n";}
};
```

y是基类保护，所以在派生类中可以直接引用。而在类外不可直接引用。

```
class B:private A{
    int m,n;

public:  B(int a,int b,int c,int d,int e):A(a,b,c){m=d;n=e;}
    void Show(){cout<<"m="<<m<<"\t"<<"n="<<n<<"\n";
    cout<<"x="<<Getx()<<"\t"<<"y="<<y<<"\t"<<"z="<<z<<"\n"; }
    int Sum(){return ( Getx()+y+z+m+n);}
};
```

私有派生

对基类初始化

z是基类公有，私有派生变为私有，所以在派生类中可直接引用，而在类外不可。

因为x是基类私有，所以在派生类和类外中不能直接引用

```
void main(void)
{  B b1(1,2,3,4,5);  A a1(1,2,3); a1.ShowA();
  b1.ShowA();        b1.Show();
```

这些函数都是基类公有，在类外不可使用。

```
  cout<< "Sum="<<b1.Sum()<<"\n";cout<<"x="<<b1.Getx()<<"\t";
  cout << "y="<<b1.Gety()<<"\t";        cout << "z="<<b1.z<<"\n";}
```



保护派生

保护派生，派生类中基类公有和保护成员降级使用

```
class ClassName: protected BaseClassName
```

保护派生时，基类中公有成员和保护成员在派生类中均变为保护成员，在派生类中仍可直接使用这些成员，基类中的私有成员，在派生类中不可直接使用。

基类成员属性	派生类	派生类外
公有	可以引用	不可引用
保护	可以引用	不可引用
私有	不可引用	不可引用

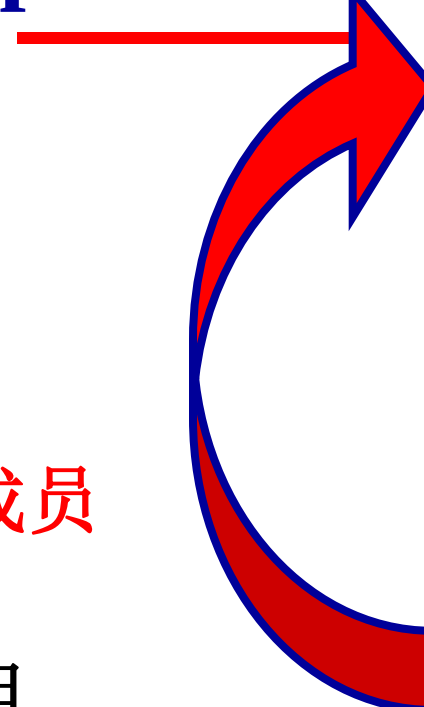
A

x(私有)
y(保护)
z(公有)
A()
Getx()
Gety()
ShowA()

B

x(私有)
y(保护)
z(保护)
A()
Getx()
Gety()
ShowA()
m(私有)
n(私有)
B()
Show()
Sum()

protected



均为保护
类B 外不
能引用



x在类B新增加的成员
中不能直接调用

y在类B中可以调用

z在类B中可以调用

对类B的对象初始化即是对x,y,z,m,n等全部成员的初始化

protected 成员是一种具有血缘关系内外有别的成员。它对派生类的对象而言，是公开成员，可以访问，对**血缘外部而言，与私有成员一样被隐蔽。**

抽象类与保护的成员函数

当定义了一个类，这个类只能用作基类来派生出新的类，而不能用这种类来定义对象时，称这种类为抽象类。当对某些特殊的对象要进行很好地封装时，需要定义抽象类。

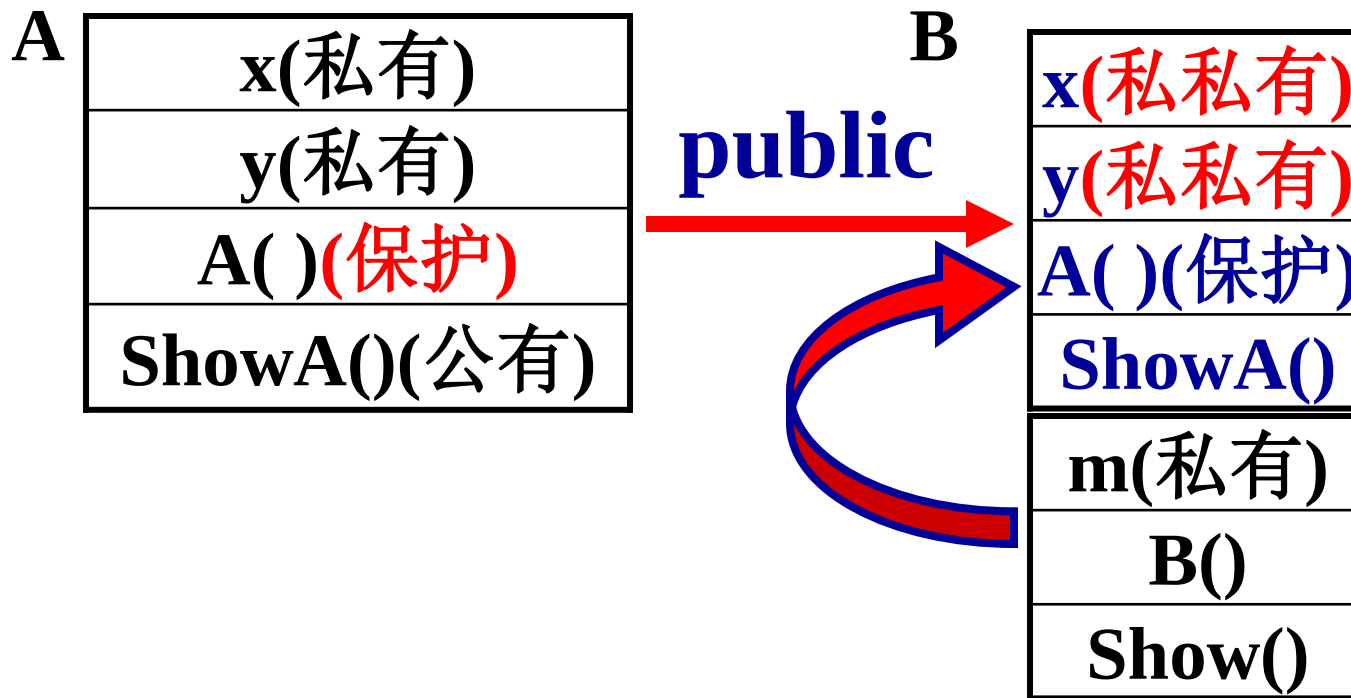
抽象类在C++中扮演接口的角色，描述了类的行为和功能，而不需要完成类的特定实现。

如果类中至少有一个函数被声明为纯虚函数，则这个类就是抽象类。

抽象类一般将类的构造函数或析构函数的访问权限定义为保护

当把类中的构造函数或析构函数说明为**私有**的时，所定义类通常是没有任何实用意义的，一般情况下，不能用它来产生对象，也不能用它来产生派生类。

“单例模式 (Singleton Pattern) ”



在类B中不能定义A的对象

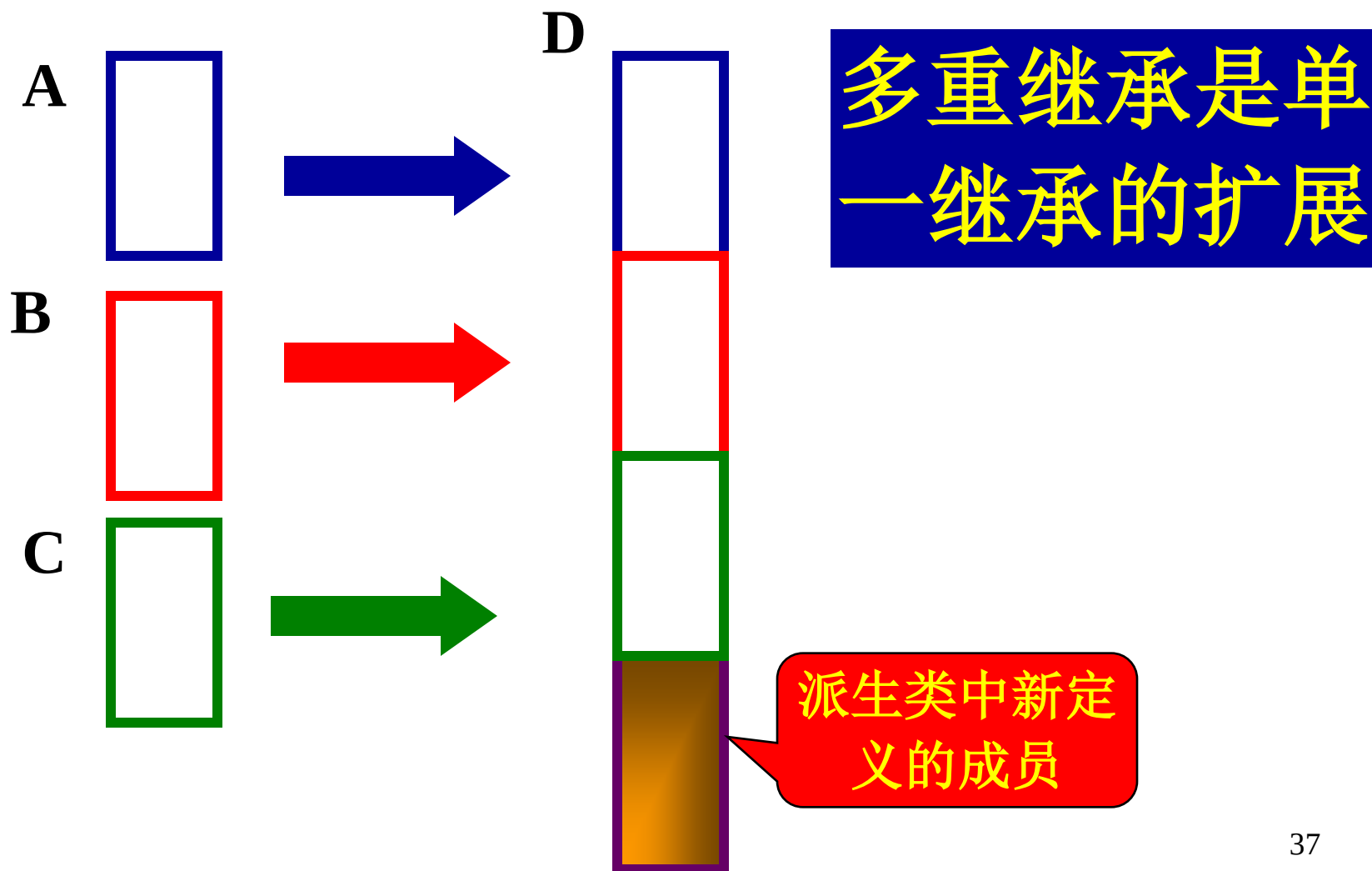
在任何时候都不能定义A的对象

但可以在初始化类B的对象时初始化原类A中的成员，
因为A()在类B中是可以被调用的。

```
class A { int x, y;  
protected: A(int a,int b){x=a;y=b;}//基类初始化  
public:  
void ShowA(){cout<< "x="<<x<<"\t"<<"y="<<y<<"\n";}  
};  
class B: public A{  
    int m;  
    A a1; //在派生类中也不可以定义A的对象，实际上还是类外调用  
public:  
    B(int a,int b,int c):A(a,b)//可以在派生类中调用A的构造函数  
        {m=c;}  
    void Show(){    cout<<"m="<<m<<"\n";    ShowA(); }  
};  
void main(void)  
{    B b1(1,2,3); //可以定义派生类对象  
    b1.Show();  
    A aa;    //不可定义A的对象    }
```

多重继承

可以用多个基类来派生一个类。



格式为:

继承方式

```
class 类名:<Access>类名1,..., <Access>类名n
{
```

```
    private:    ..... ;    //私有成员说明;
```

```
    public:     ..... ;    //公有成员说明;
```

```
    protected: ..... ;    //保护的成员说明;
```

```
};
```

```
class D: public A, protected B, private C
```

```
{    ....//派生类中新增加成员
```

```
};
```

```

class A{ int x1,y1;
public:  A(int a,int b)    { x1=a; y1=b;    }
void ShowA(void){ cout<<"A.x="<<x1<<"\t"<<"A.y="<<y1<<endl; }
};

class B{int x2,y2;
public:  B(int a,int b)    {x2=a; y2=b;    }
void ShowB(void){ cout<<"B.x="<<x2<<"\t"<<"B.y="<<y2<<endl; }
};

class C:public A,private B{
    int x,y;
public:  C(int a,int b,int c,int d,int e,int f):A(a,b),B(c,d)    {x=e; y=f;    }
void ShowC(void){cout<<"C.x="<<x<<"\t"<<"C.y="<<y<<endl;
                ShowA();ShowB();    }
};

void main(void)
{
    C c(1,2,3,4,5,6);
    c.ShowC();
    c.ShowA ();
    c.ShowB ();
}

```

公有派生 (Public Inheritance)

私有派生 (Private Inheritance)

仍为公有 (Still Public)

成为私有 (Becomes Private)

非法，私有类外不可调用 (Illegal, cannot call private method from outside the class)

B b1(10,20);

b1.ShowB();

初始化基类成员

- 构造函数不能被继承
- 派生类的构造函数必须调用基类的构造函数来初始化基类成员以及基类子对象。

派生类构造函数的调用顺序如下：

基类的构造函数

子对象类的构造函数

派生类的构造函数

```
class B:public A{
```

```
    int y;
```

```
    A a1;
```

```
public:
```

```
    B(int a, int b):A(a),a1(3){y=b;}
```

```
    .....
```

```
};
```

基类的构造函数

子对象类的构造函数

派生类的构造函数

当撤销派生类对象时，析构函数的调用正好相反。


```

class Base1 { int x;
public: Base1(int a){x=a;cout<<"调用基类1的构造函数!\n"; }
      ~Base1( ){    cout<<"调用基类1的析构函数!\n";    }
};

class Base2 { int y;
public: Base2(int a){ y=a;cout<<"调用基类2的构造函数!\n"; }
      ~Base2( ){    cout<<"调用基类2的析构函数!\n";    }
};

class Derived:public Base1, public Base2{
    int z;
public: Derived(int a,int b):Base1(a),Base2(20)
    {z=b; cout<<"调用派生类的构造函数!\n";}
    ~Derived( ){cout<<"调用派生类的析构函数!\n";}
};

void main(void)
{
    Derived c(100,200);
}

```

调用基类1的构造函数

调用基类2的构造函数

调用派生类的构造函数

调用派生类的析构函数

调用基类2的析构函数

调用基类1的析构函数

```
class Base1 { int x;
public: Base1(int a){x=a;cout<<"调用基类1的构造函数!\n"; }
      ~Base1( ){    cout<<"调用基类1的析构函数!\n";    }
};
```

```
class Base2 { int y;
public: Base2(int a){ y=a;cout<<"调用基类2的构造函数!\n"; }
      ~Base2( ){    cout<<"调用基类2的析构函数!\n";    }
};
```

先说明基类2

```
class Derived:public Base2, public Base1{
    int z;
public: Derived(int a,int b):Base1(a),Base2(20)
    {z=b; cout<<"调用派生类的构造函数!\n";}
~Derived( ){cout<<"调用派生类的析构函数!\n";}
};

void main(void)
{    Derived  c(100,200);
}
```

调用基类2的构造函数

调用基类1的构造函数

调用派生类的构造函数

调用派生类的析构函数

调用基类1的析构函数

调用基类2的析构函数

```

class Base1 { int x;
public: Base1(int a){x=a;cout<<"调用基类1的构造函数!\n"; }
      ~Base1( ){    cout<<"调用基类1的析构函数!\n";    }
};

class Base2 { int y;
public: Base2(int a){ y=a;cout<<"调用基类2的构造函数!\n"; }
      ~Base2( ){    cout<<"调用基类2的析构函数!\n";    }
};

class Derived:public Base1, public Base2{
    int z;
public: Derived(int a,int b):Base2(a),Base1(20)
    {z=b; cout<<"调用派生类的构造函数!\n";}
~Derived( ){cout<<"调用派生类的析构函数!\n";}
};

void main(void)
{    Derived  c(100,200);
}

```

调用基类1的构造函数

调用基类2的构造函数

调用派生类的构造函数

调用派生类的析构函数

调用基类2的析构函数

调用基类1的析构函数

```
class Derived:public Base2, public Base1{
```

```
    int z;
```

基类子对象

```
    Base1 b1,b2;
```

```
public:
```

基类成员构造

基类子对象构造

```
    Derived(int a,int b):Base1(a),Base2(20), b1(200),b2(a+b)
```

```
    {z=b; cout<<"调用派生类的构造函数!\n";}
```

```
    ~Derived( ){cout<<"调用派生类的析构造函数!\n";}
```

```
};
```

```
void main(void)
```

```
{    Derived  c(100,200);
```

```
}
```

基类成员构造用基类名，
基类子对象构造用对象名

```
class Base1 { int x;
public: Base1(int a){x=a;cout<<"调用基类1的构造函数!\n"; }
      ~Base1( ){    cout<<"调用基类1的析构函数!\n";    }
};

class Base2 { int y;
public: Base2(int a){ y=a;cout<<"调用基类2的构造函数!\n"; }
      ~Base2( ){    cout<<"调用基类2的析构函数!\n";    }
};

class Derived:public Base2, public Base1{
      int z;  Base1 b1,b2;
public: Derived(int a,int b):Base1(a),Base2(20), b1(200),b2(a+b)
      {z=b; cout<<"调用派生类的构造函数!\n";}
~Derived( ){cout<<"调用派生类的析构函数!\n";}
};

void main(void)
{      Derived c(100,200);
}
```

调用基类2的构造函数
调用基类1的构造函数
调用基类1的构造函数
调用基类1的构造函数
调用派生类的构造函数
调用派生类的析构函数
调用基类1的析构函数
调用基类1的析构函数
调用基类1的析构函数
调用基类2的析构函数

说明基类1的对象
b1,b2

```

class Base1 {
    int x;
public: Base1(int a);
    ~Base1( );
};

class Base2 {
    int y;
public: Base2(int a);
    ~Base2( );
};

class Derived:public Base1, public Base2{
    int z; Base1 b1,b2;
public: Derived(int a,int b);
    ~Derived( );
};
    
```

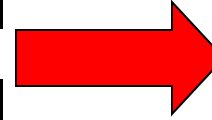
Base1

x
Base1()
~Base1()

Base2

y
Base2()
~Base2()

Derived

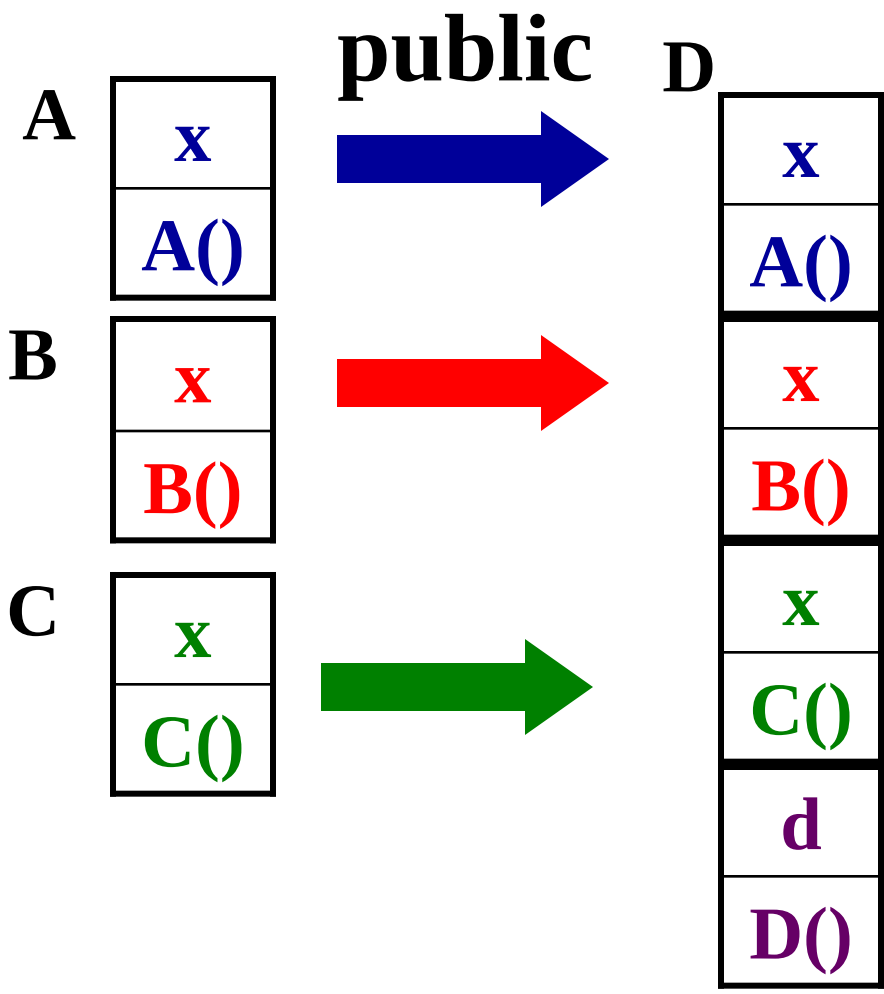


b1

b2

x
Base1()
~Base1()
y
Base2()
~Base2()
x
Base1()
~Base1()
x
Base1()
~Base1()
z
Derived()
~Derived

冲突



派生类对象

`D v;`

`v.x=5;`

产生了冲突

用类作用符限定

`v.A::x=5;`

```
class A{
public:  int x;
    void Show(){cout <<"x="<<x<<"\n";}
    A(int a=0){x=a;}
};
```

```
class B{
public:  int x;
    void Show(){cout <<"x="<<x<<"\n";}
    B(int a=0){x=a;}
};
```

```
class C:public A,public B{  int y;
public: void Setx(int a){ x=a;} //c1对象中有两个x成员
        void Sety(int b){y=b;}
        int Gety() {return  y;}
};
```

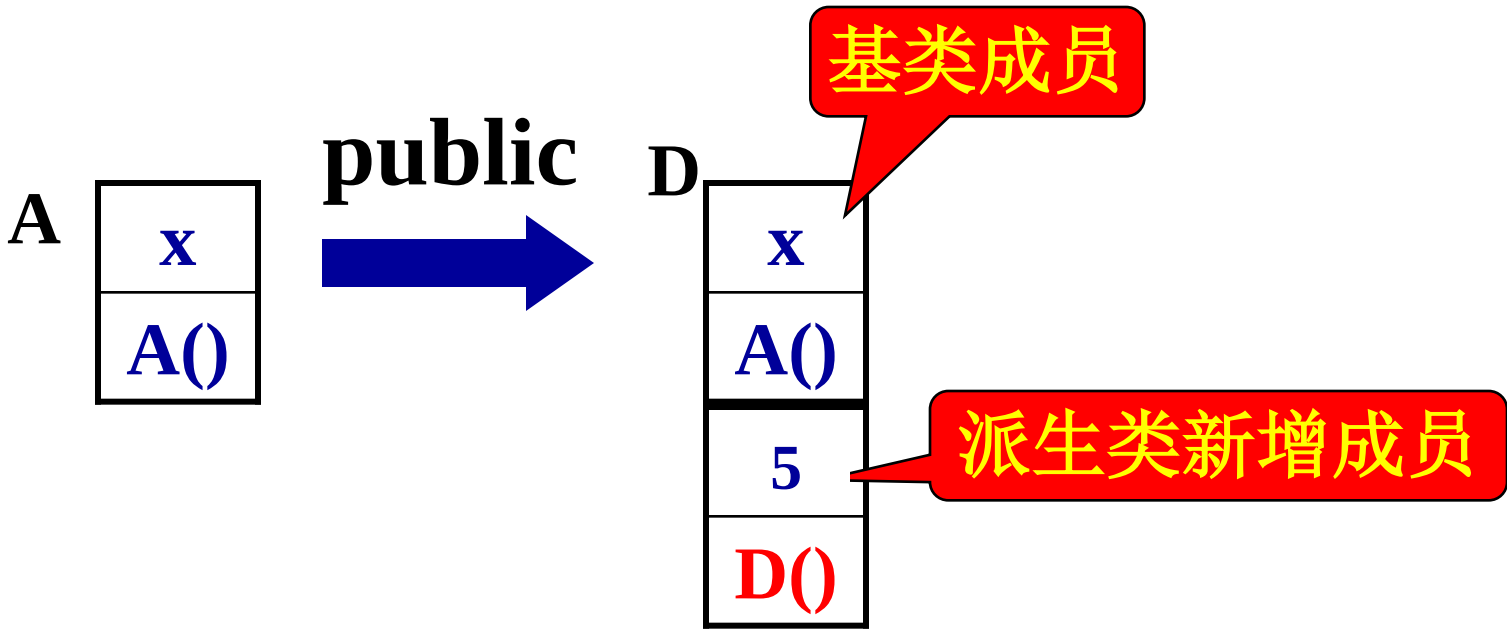
```
void main(void)
{
    C c1; c1.Show(); //c1对象中有两个Show()函数
}
```

这时，可以利用类作用域符::来指明数据或函数的来源。

如: **A::x=a;**

c1.B::Show();

支配规则



`D v;`
`v.x=5;`

当派生类中新增加的数据或函数与基类中原有的同名时，若不加限制，则优先调用派生类中的成员。

```
class A{
public: int x;
    void Show(){cout <<"x="<<x<<"\n";}
};
class B{
public: int y;
    void Show(){cout <<"y="<<y<<"\n";}
};
class C:public A,public B{
public: int y; //类B和类C均有y的成员
};
void main(void)
{ C c1;      c1.x=100;
  c1.y=200;   //给派生类中的y赋值
  c1.B::y=300; //给基类B中的y赋值
  c1.A::Show();
  c1.B::Show(); //用作用域运算符限定调用的函数
  cout <<"y="<<c1.y<<"\n";      //输出派生类中的y值
  cout <<"y="<<c1.B::y<<"\n";  //输出基类B中的y值
}
```

当派生类中新增加的数据或函数与基类中原有的同名时，若不加限制，则优先调用派生类中的成员。


```

class A{
public: int x;
      A(int a=0) { x=a;}
};

class B{
public: int y;
      B(int a=0) { y=a;}
};

class C:public A{
      int z;   B  b1; A  a1;
public: C(int a,int b,int m):A(a),b1(b),a1(a+b){ z=m; }
      void Show( ){
        cout<<"x="<<x<<"\t";
        cout<<"y="<<b1.y<<"\t";
        cout<<"x="<<z<<"\n";
        cout<<"a1.x="<<a1.x<<endl;    }
};

void main(void)
{
    C c1(100,200,500);
    c1.Show();}

```

从基类A中继承

x

在类C中新说明

z

在类C中新说明

Show()

在类C中新说明

a1.x

在类C中新说明

b1.y

C的对象所占空间

从基类A中继承

新说明类A对象中的x

基类与对象成员

任一基类在派生类中只能继承一次，否则，会造成成员名的冲突

把一个类作为派生类的基类或把一个类的对象作为一个类的成员，在使用上是有区别的：在派生类中可直接使用基类的成员（访问权限允许的话），但要使用对象成员的成员时，必须在对象名后加上成员运算符“.”和成员名。

在平面上作两个点，连一直线，求直线的长度和直线中点的坐标。

基类为**Dot**，有两个公有数据成员，即平面上的坐标 (x,y) ，同时有构造函数及打印函数。

派生类为**Line**，有两个基类**Dot**对象，分别存放两点的坐标，同时，从基类继承了一个**Dot**数据，存放直线中点的坐标。

x
y
Dot(x,y)(构造)
Dot(&dot)(拷贝)
Show()

Dot的对象空间

**Line
对象
空间**

x(中点)	
y(中点)	
Dot(x,y)	
Dot(&dot)	
Show()	
d1	x
	y
	Dot(x,y)
	Dot(&dot)
	Show()
d2	x
	y
	Dot(x,y)
	Dot(&dot)
	Show()
Line(dot1,dot2)	
Showl()	

**从基类
继承**

**基类
对象**

```
class Dot{
public: float x,y;
      Dot(float a=0,float b=0){ x=a; y=b;}
      void Show(void){cout<<"x="<<x<<"\t"<<"y="<<y<<endl;}
};
```

用坐标初始化Dot对象

打印坐标

```
class Line:public Dot{
      Dot d1,d2;
public: Line(Dot dot1,Dot dot2):d1(dot1),d2(dot2)
      { x=(d1.x+d2.x)/2; y=(d1.x+d2.y)/2;}
      void Showl(void){ cout<<"Dot1: "; d1.Show(); cout<<"Dot2: "; d2.Show();
      cout<<"Length="<<sqrt((d1.x-d2.x)*(d1.x-d2.x)+(d1.y-d2.y)*(d1.y-d2.y))<<endl;
      cout<<"Center: " <<"x="<<x<<"\t"<<"y="<<y<<endl;    }
};

void main(void)
{ float a,b;
  cout<<"Input Dot1: \n"; cin>>a>>b;
  Dot dot1(a,b);//调用Dot的构造函数
  cout<<"Input Dot2: \n"; cin>>a>>b;
  Dot dot2(a,b);      Line line(dot1,dot2); line.Showl();}
```

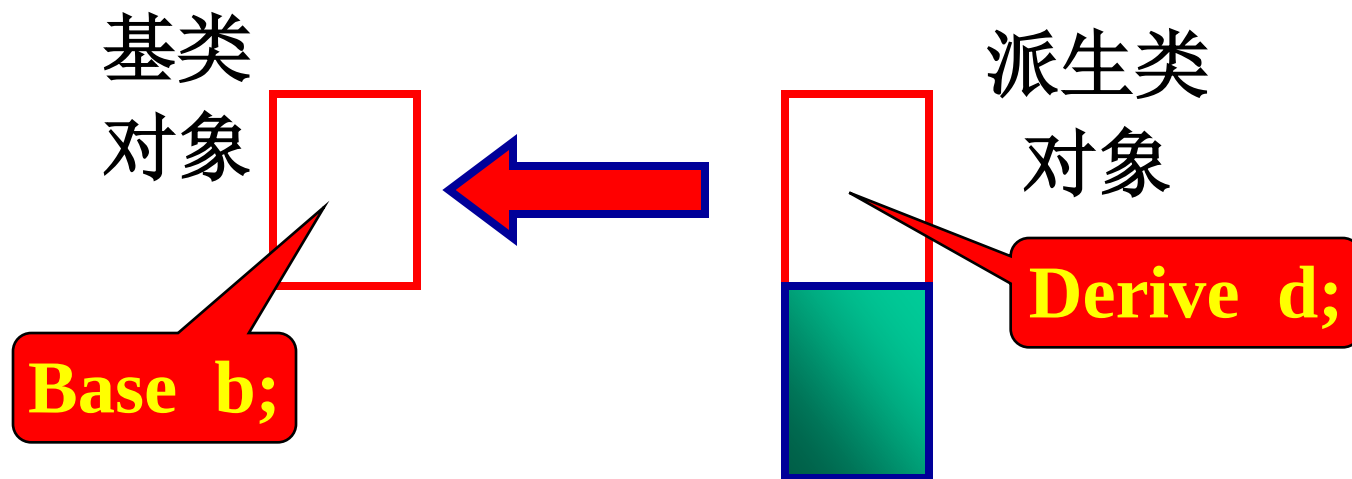
在Line中新说明的成员

对成员初始化

x, y是继承基类的成员

赋值兼容规则

相互之间能否赋值？



可以将派生类对象的值赋给基类对象。反之不行

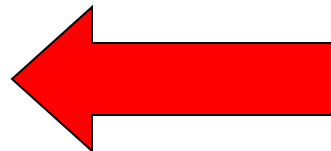
b = d;



只是将从基类继承来的成员赋值。

x
y
Dot(x,y)
Dot(&dot)
Show()

Dot的对象空间



Line的对象空间

```
Dot dot;
Line line;
dot=line;
line=dot;
```

非法

x	
y	
Dot(x,y)	
Dot(&dot)	
Show()	
d1	x
	y
	Dot(x,y)
	Dot(&dot)
	Show()
d2	x
	y
	Dot(x,y)
	Dot(&dot)
	Show()
Line()	
Showl()	

从基类继承

基类对象

可以将一个派生类对象的地址赋给基类的指针变量。

基类指针

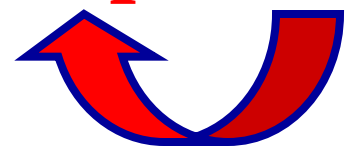
派生类对象

basep只能引用
从基类继承来
的成员。

Base *basep;

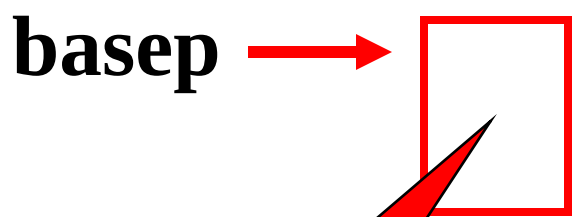
basep=&b;

basep = &d



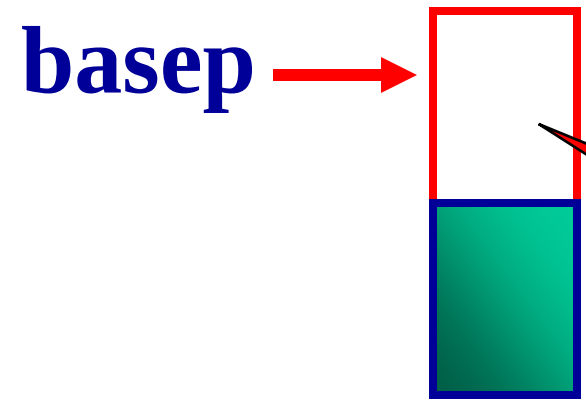
基类对象

基类对象



Base b;

派生类对象



Derive d;

派生类对象可以初始化基类的引用。

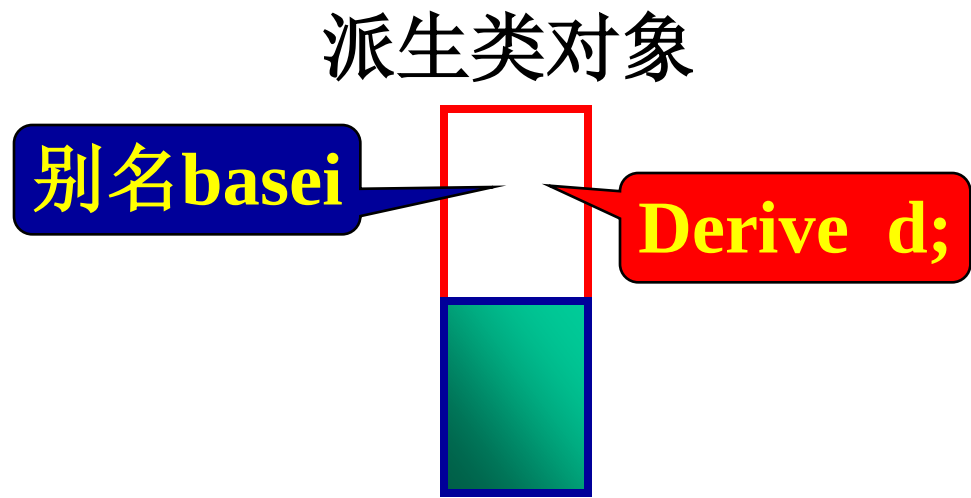
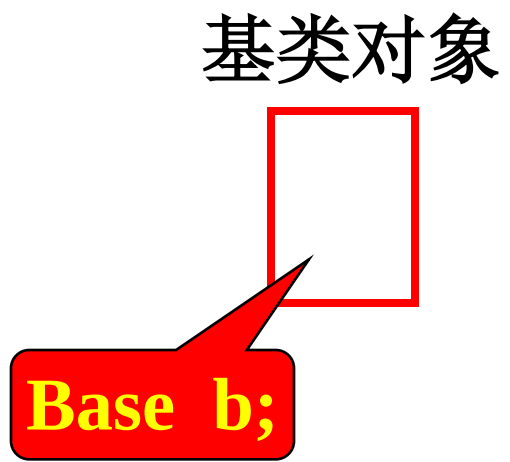
Derive d;

派生类对象

Base basei=&d;

基类引用

basei只能引用
从基类继承来
的成员。



```
class A{
```

```
public: int x;
```

```
    A(int a=0) { x=a;}
```

```
};
```

```
class B{
```

```
public: int y;
```

```
    B(int a=0) { y=a;}
```

```
};
```

```
class C:public A,public B{    int z;
```

```
public: C(int a,int b,int m): A(a), B(b) { z=m; }
```

```
    void Show() {    cout<<"x="<<x<<"\t";    cout<<"y="<<y<<"\t";
```

```
    cout<<"z="<<z<<"\n";    }
```

```
};
```

```
void main(void)
```

```
{    A  a1(100);    B  b1(200);    C  c1(10,20,50);
```

```
    cout<<"a1.x="<<a1.x<<endl;    cout<<"b1.y="<<b1.y<<endl;
```

```
    c1.Show();    a1=c1; b1=c1; //派生类对象向基类对象赋值
```

```
    cout<<"a1.x="<<a1.x<<endl;    cout<<"b1.y="<<b1.y<<endl;
```

```
    A *p;    p=&c1; //基类指针指向派生类对象    p->Show();
```

```
}
```

a1.x=100

b1.y=200

x=10 y=20 z=50

a1.x=10

b1.y=20

a1

10
A()

b1

20
B()

c1

x 10
A()
y 20
B()
z 50
C()
Show()

错误!

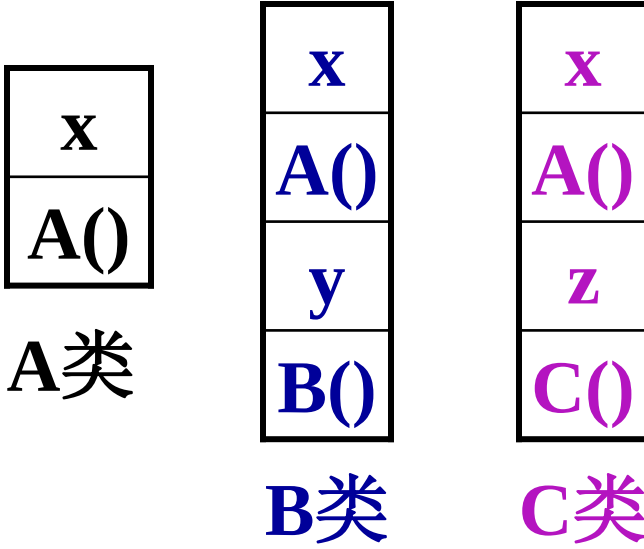
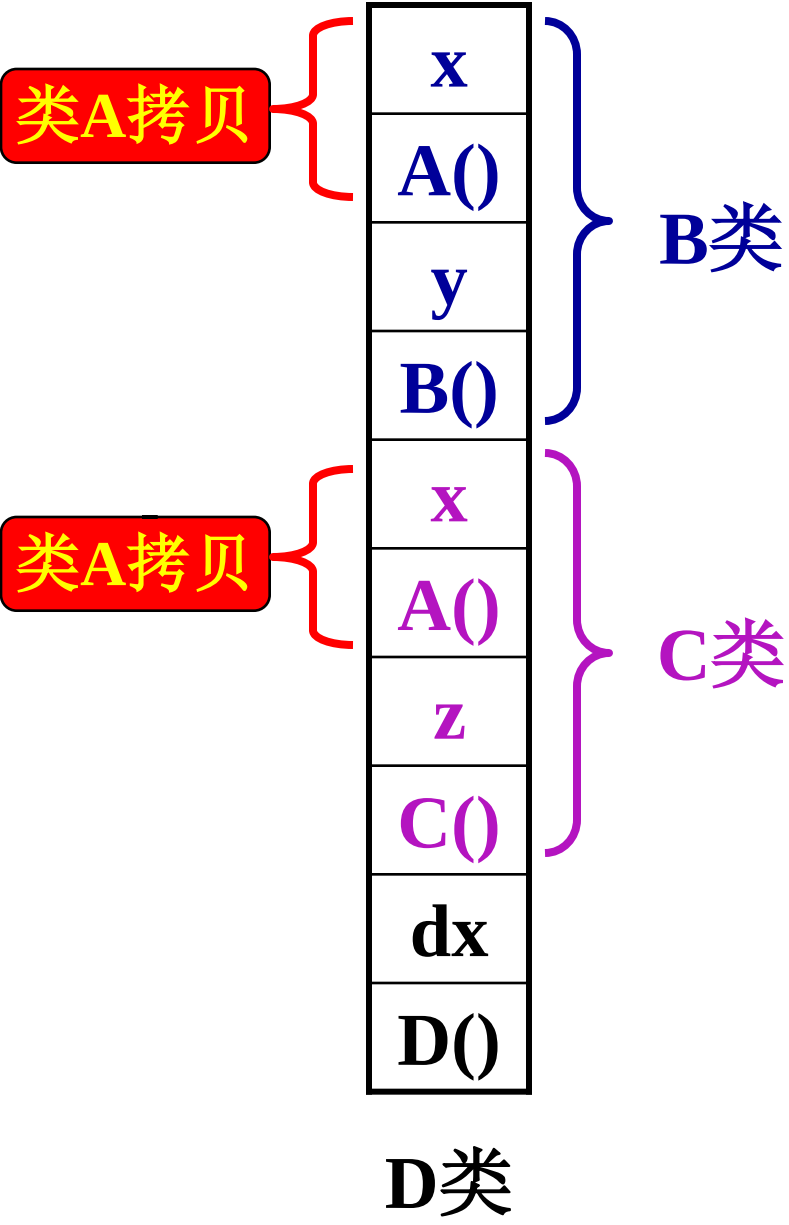
虚基类

类B 是类A的派生类

类C 是类A的派生类

类D 是类B和类C的派生类

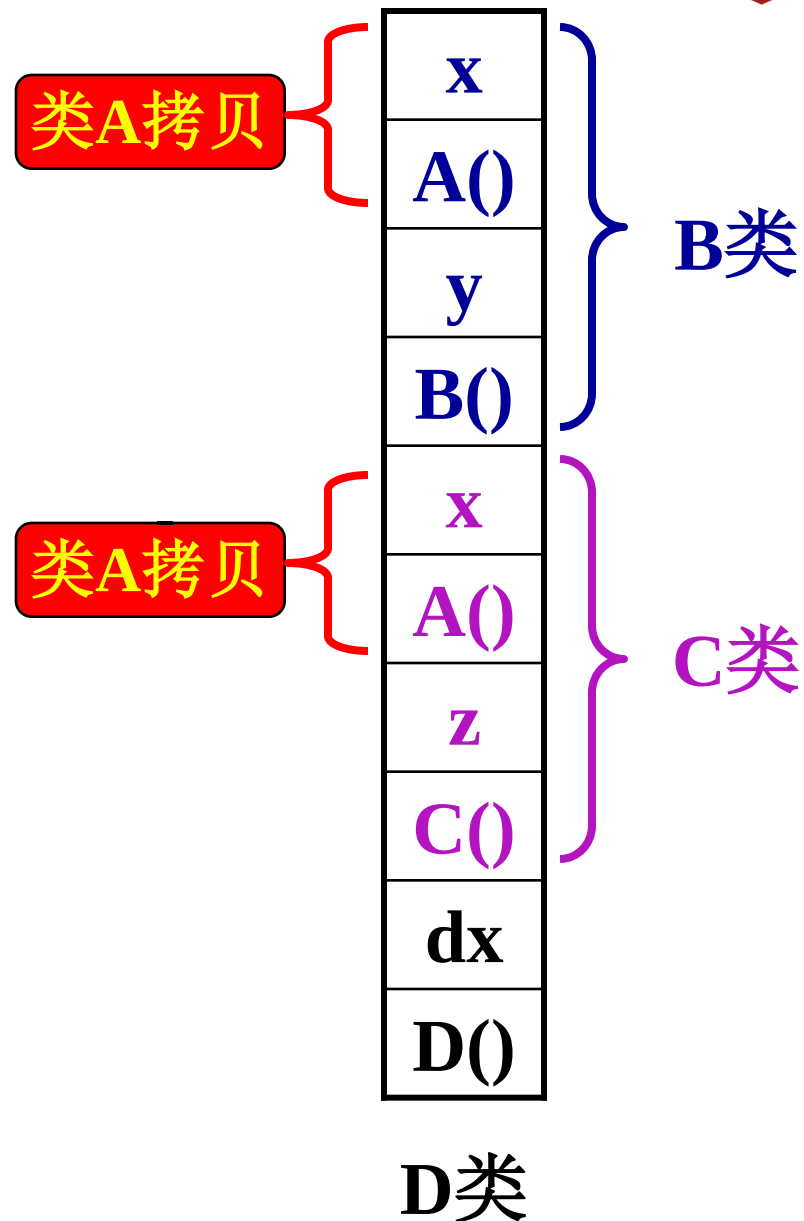
这样，类D中就有两份类A的拷贝



这种同一个公共的基类在派生类中产生多个拷贝，不仅多占用了存储空间，而且可能会造成多个拷贝中的数据不一致 和模糊的引用。

D d;

d.x=10; //模糊引用



```

class A{
public: int x;
      A(int a=0) { x=a;}
};

class B:public A{
public: int y;
      B(int a=0,int b=0):A(a) { y=b;}
};

class C:public A{
public: int z;
      C(int a,int c):A(a){ z=c; }
};

class D:public B,public C{
public: int dx;
      D(int a1,int b,int c,int d,int a2):B(a1,b),C(a2,c)
      {
          dx=d;}
};

void main(void)
{
    D d1(10,20,30,40,50);    cout<<d1.x<<endl; }

```

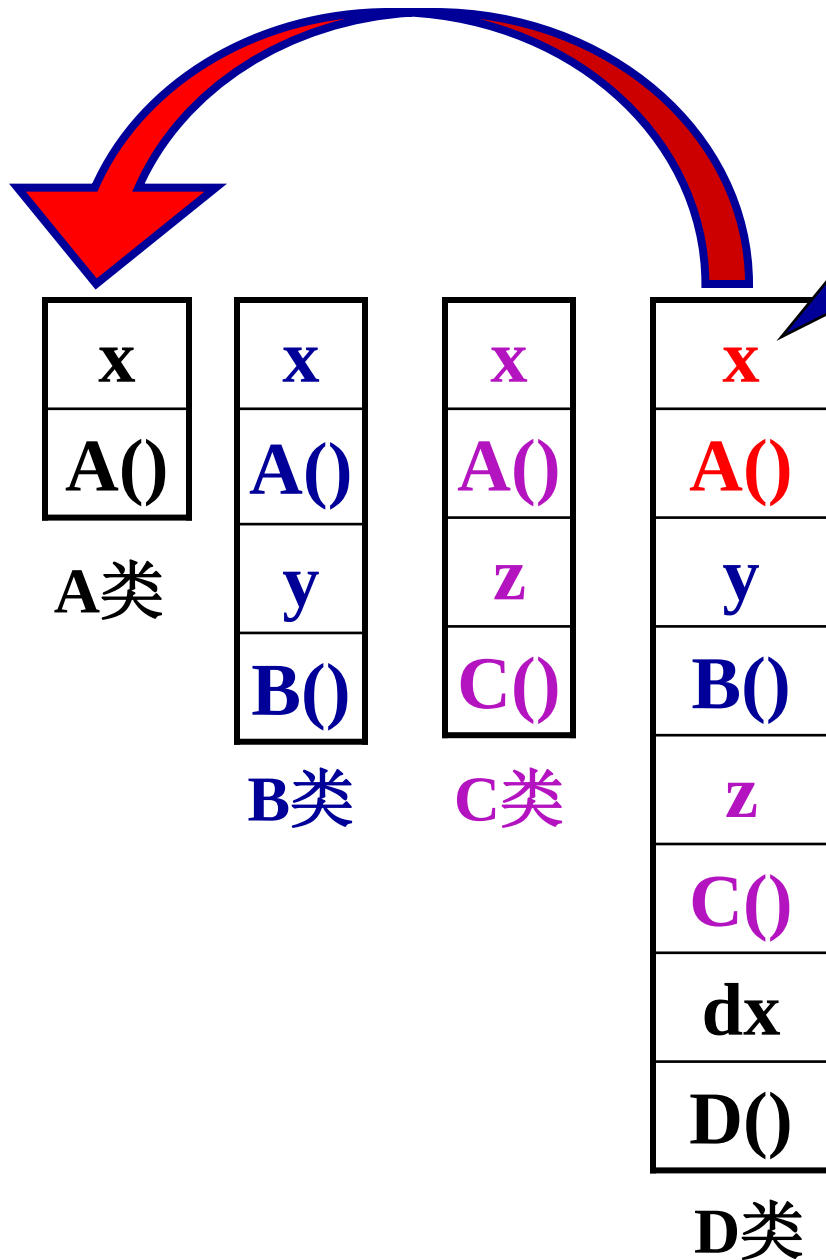
模糊引用，错误！

X	a1=10
A()	
y	b=20
B()	
x	a2=50
A()	
z	c=30
C()	
dx	dx=40
D()	
D类	

在多重派生的过程中，若使公共基类在派生类中只有一个拷贝，则可将这种基类说明为**虚基类**。

在派生类的定义中，只要在基类的类名前加上关键字**virtual**，就可以将基类说明为虚基类。

```
class B:public virtual A{  
  
public:  
  
    int y;  
  
    B(int a=0, int b=0 ):A(b) { y=a;}  
  
};
```



一份拷贝，在D()的构造函数中直接调用A()

由虚基类派生出的对象初始化时，**直接调用**虚基类的构造函数。因此，若将一个类定义为虚基类，**则一定有正确的构造函数**可供所有派生类调用。

```
class A{
public: int x;
    A(int a=0) { x=a;}
};

class B:public virtual A{
public: int y;
    B(int a=0,int b=0): A(a) {y=b;}
};

class C:public virtual A{
public: int z;
    C(int a=0,int c=0):A(a){z=c; }
};

class D:public B,public C{
public: int dx;
    D(int a1,int b,int c,int d,int a2):B(a1,b),C(a2,c) , A(a2)
    {
        dx=d;}
};
```

```
void main(void)
{
    D d1(10,20,30,40,50);
    cout<<d1.x<<endl;
    d1.x=400;
    cout<<d1.x<<endl;
    cout<<d1.y<<endl;
}
```

50	0
400	400
20	20

直接在派生类中调用虚基类的构造函数

没有对虚基类构造函数的调用，用缺省的构造函数

再次强调，用虚基类进行多重派生时，若虚基类没有缺省的构造函数，则在每一个派生类的构造函数中都必须有对虚基类构造函数的调用。

继承：进一步总结和说明

继承：进一步总结和说明

■ 继承和派生

- 继承允许以现有的类为基础来构建新类
- 新类（派生类）继承现有类（基类）的属性和行为
- 派生类可以修改（改变权限或重定义）基类的属性和行为
- 派生类可以增加新的属性和行为
- 派生类对象也是基类对象
- 派生类对象和基类对象可以被统一管理

例子：定义基类Mammal和派生类Lion

```
class Mammal
{public:
    Mammal();
    ~Mammal();
    .....
};
class Lion: public Mammal
{public:
    Lion();
    ~Lion();
    .....
};
```

继承：进一步总结和说明

- 继承的三种方式
 - 公有继承 (public)
 - 受保护继承 (protected)
 - 私有继承 (private)
 - 在不同方式下，基类中的成员被派生类继承后对外的可见性会有所不同

继承：进一步总结和说明

■ 类的层次

- 一个类可以是某个继承关系中的基类，也可以是另一个继承关系中的派生类
- 类A派生出类B，类B又派生出类C，则类B是类C的直接基类，类A是类C的间接基类
- 定义派生类时，直接基类要明确列出，间接基类不用列出

例子：间接基类的例子

```
class A
{protected:
    int aMember; .....
};
class B: protected A
{protected:
    int bMember; .....
};
class C: private B
{protected:
    int cMember; .....
};
```

继承：进一步总结和说明

- 在派生类中重定义基类的成员函数
 - 派生类以某种方式继承基类中的所有成员（构造函数、析构函数的使用有特殊规定）
 - 如果基类成员函数功能不满足需要——在派生类中可以用同一函数名、相同的参数表重定义该函数（注意：和函数重载不同）

继承：进一步总结和说明

- 派生类对数据成员和成员函数的使用（一）
 - 在派生类成员函数中，可以直接访问自己定义的数据成员和从基类继承的异名非私有数据成员，但只能以 **基类名 :: 数据成员名** 的方式访问定义于基类的同名非私有数据成员；可以直接调用自己定义的成员函数和从基类继承的异名非私有成员函数，但只能以 **基类名 :: 函数名** 的方式调用定义于基类的同名非私有成员函数（包括重定义和重载两种情况）

继承：进一步总结和说明

- 派生类对数据成员和成员函数的使用（二）
 - 在外部函数中，可以通过派生类对象名（对象名.）使用自己定义的接口成员和从基类继承的异名接口成员，但只能以 对象名.基类名 :: 接口成员名 的方式使用定义于基类的同名接口成员

继承：进一步总结和说明

- 类指针操作类对象的几种可能
 - 基类指针操作基类对象(自然)
 - 派生类指针操作派生类对象(自然)
 - 基类指针操作派生类对象(只能访问基类成员)——把派生类对象作为基类对象看(安全)
 - 派生类指针操作基类对象——把基类对象作为派生类对象看(危险, 禁用)

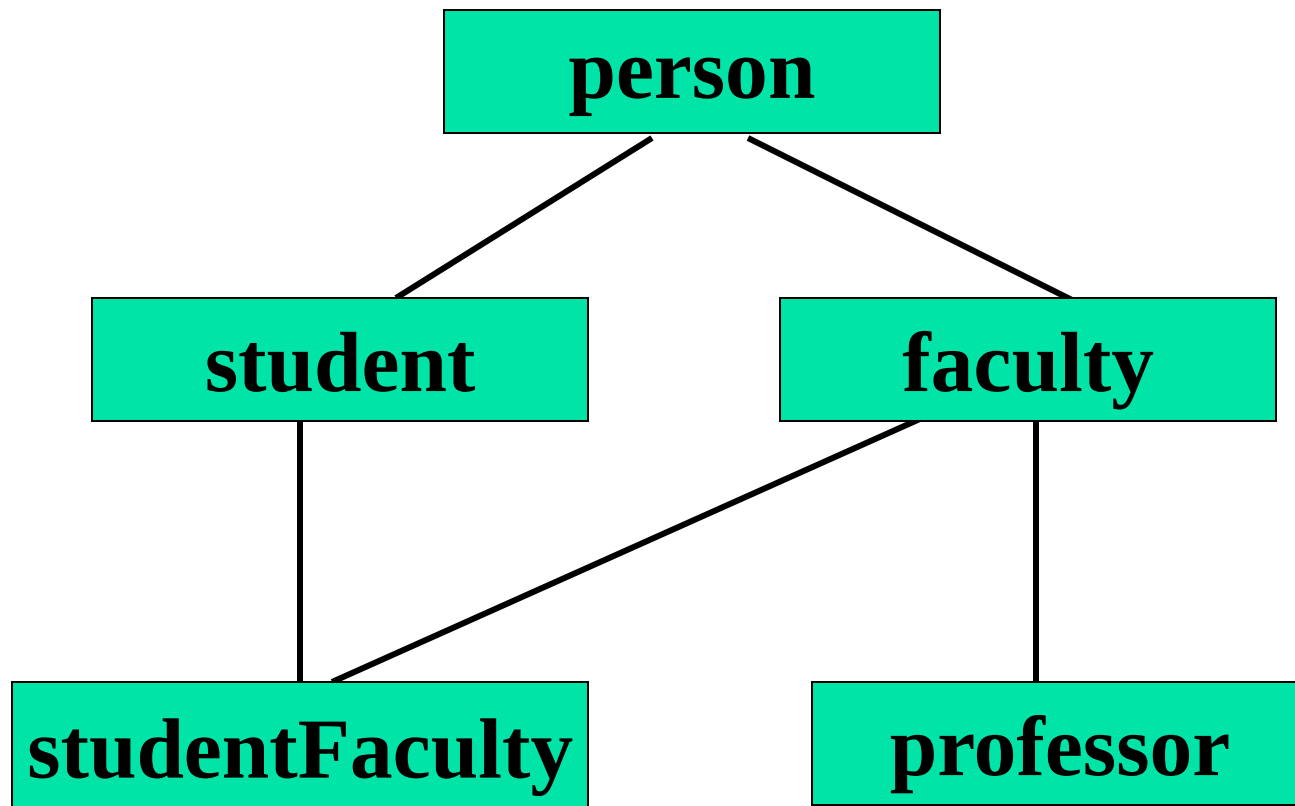
继承：进一步总结和说明

- 基类的构造函数和析构函数是派生类的特殊成员函数，它们既不是派生类的构造函数和析构函数，也不能当作一般成员函数使用
 - 派生类的构造函数在构造派生类对象时需要调用基类的构造函数对其中定义于基类的数据成员进行初始化
 - 需要传递参数时，显式调用
 - 不需要传递参数时，隐式调用
 - 基类的析构函数在删除派生类对象时被隐式调用。
- C++在派生类析构函数的函数体末尾自动插入：
~直接基类名();

继承：进一步总结和说明

- 派生类构造函数的执行顺序
 - 先调用基类的构造函数初始化从基类继承的数据成员
 - 再执行自己的函数体初始化定义于派生类的数据成员
- 派生类析构函数的执行顺序
 - 派生类的析构函数先执行自己的函数体
 - 再调用基类的析构函数

示例：大学人员的继承层次关系



大学人员相关类的定义和实现 (1/7)

```
#include <iostream>
#include <string.h>
#include <assert.h>

char *copy(char *s)
{char *temp=new char[strlen(s)+1];
  assert(temp!=0);
  strcpy(temp,s);
  return temp;
}
```

大学人员相关类的定义和实现 (2/7)

```
class person
{protected:
    char *name;
    int age;
    person(char *n,int a)
        {name=copy(n);age=a;}
    person( ) { }

    ~person( )
        {delete [] name;}
};
```

大学人员相关类的定义和实现(3/7)

```
class student:virtual public person
{private:
    char *major;
protected:
    student(char *m) {major=copy(m) ;}
public:
    student(char *n,int a,char *m)
        :person(n,a)
        {major=copy(m) ;}
    ~student() {delete [] major;}
    void print()
        {cout<<name<<endl<<"Age: "
            <<age<<" Major: "<<major<<endl ;}
};
```

大学人员相关类的定义和实现 (4/7)

```
class faculty:virtual public person
{protected:
    char *dept;
    faculty(char *d) {dept=copy(d) ;}
public:
    faculty(char *n,int a,char *d)
        :person(n,a)
        {dept=copy(d) ;}
    ~faculty() {delete [] dept;}
    void print( )
        {cout<<name<<endl<<"Age: " <<age
            <<"Department: " <<dept<<endl;
        }
};
```


大学人员相关类的定义和实现 (5/7)

```
class professor:public faculty
{private:
    int level;
public:
    professor(char *n,int a,char *d,int h)
        :person(n,a),faculty(d)
        {level=h;}
    void print( )
        {faculty::print( );
         cout<<"Level: "<<level<<endl;
        }
};
```

大学人员相关类的定义和实现 (6/7)

```
class studentFaculty:public student,
                      public faculty
{public:
    studentFaculty(char *n,int a,char *m,
                  char *d)
        :person(n,a) , student(m) , faculty(d)
    {    }
    void print( )
    {student::print( ) ;
      cout<<"Department: " <<dept<<endl;
    }
};
```

大学人员相关类的定义和实现 (7/7)

```
main()  
{  
    student  
        myStudent("Zhang", 25, "Computer");  
    faculty  
        myFaculty("Wang", 32, "Engineering");  
    professor  
        myProfessor("Li", 50, "Management", 2);  
    studentFaculty myStudentFaculty  
("Zhao", 23, "English", "Foreign Language");  
    myStudentFaculty.print();  
    myProfessor.print();  
    myFaculty.print();  
    myStudent.print();  
    return 0;  
}
```

程序运行结果:

Zhao

Age: 23 Major: English

Department: Foreign Language

Li

Age: 50 Department: Management

Level: 2

Wang

Age: 32 Department: Engineering

Zhang

Age: 25 Major: Computer

构造函数的误用

```
main()  
{  
    student  
        myStudent("Computer"); ✗  
    faculty  
        myFaculty("Engineering"); ✗  
    person myPerson("Li", 50); ✗  
    student  
        myStudent1("Zhang", 25, "Computer");  
    person myPerson1(myStudent1); ✓  
    return 0;  
}
```

继承：进一步总结和说明

- 如果在多条继承路径上有一个公共的基类，多重继承的派生类对公共基类的同一数据成员就可能产生多个拷贝
 - 把公共基类作为虚基类继承
 - 派生类的构造函数需小心设计

学习目的检测

- 理解继承的概念
- 掌握如何通过继承建立类的层次结构
- 掌握通过类指针操作继承关系中的对象的方法
- 了解多重继承

Thank you !